database). EF Core works with many databases, including the popular Microsoft SQL Server and MySQL database systems. This book will use EF Core with SQLite, an open source, cross-platform, file-based database. While lacking many advanced features you find in more powerful server-based systems, SQLite is easy to use, requiring no installation or configuration, and will suffice for the EF Core features we will explore in this chapter.

8.1.1 Why choose EF Core?

You could use low-level, old-school ADO.NET APIs to manage communication with the database, but the code required is repetitive (did I say that already?) and boring to write. One solution would be to write your own library of helpers to reduce the repetition. But then you would have to maintain that code yourself. Data access libraries are abundant, and unless you can see ways to improve on existing offerings, writing your own is likely to be a waste of time if you just need to get on with producing a web application.

You are free to use any library you like for managing data access in Razor Pages applications, so of all the abundant existing libraries, why choose EF Core? Being part of .NET, it is well supported and tested and enjoys a significant amount of official documentation. On top of the official docs, there are a huge number of community contributions, such as books, blog posts, and tutorial sites, that explore the more esoteric features of EF Core along with its most common workflows. If all else fails and you struggle to get something to work, you can post an issue to the EF Core GitHub repo (https://github.com/dotnet/efcore), where you might even get a response from one of the developers on the EF Core team.